

hw01_moneyball_analysis_python_final report

October 2, 2026

1 Homework 01: Moneyball Analysis

Course: DATA 621- Data Mining **Authors:** Alinzon Simon, Denise Atherley, Joao De Oliveira, Jane Song, Ciara Bonnett-Jones 10/2/2026

This notebook contains the complete analysis for the assignment. It begins by understanding the training and evaluation data, then continues through data preparation, multiple linear regression, model evaluation, and predictions.

The data represents professional baseball teams from 1871 through 2006. The assignment asks us to predict TARGET_WINS using the variables provided in the training data.

```
[1]: from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import variance_inflation_factor

pd.set_option("display.max_columns", None)
```

1.1 1. Exploratory Data Analysis

The goal is to understand the variables, check the quality of the data, and identify possible relationships with wins before fitting a model. The dependent variable is TARGET_WINS. The evaluation data does not contain that variable and will be used later for predictions. INDEX is an identification variable and should not be used as a predictor.

1.1.1 Load the datasets

The notebook is stored in homeworks/hw01/notebooks/, so the data files are one folder above in ../data/. To read the files directly from the GitHub repository over the internet, the “Raw” GitHub URLs are used. The pandas library can read CSVs directly from web addresses.

```
[2]: # Use the Raw GitHub URLs instead of local paths
training_url = "https://raw.githubusercontent.com/alinsimon/data621-data-mining/
↳main/homeworks/hw01/data/moneyball-training-data.csv"
evaluation_url = "https://raw.githubusercontent.com/alinsimon/
↳data621-data-mining/main/homeworks/hw01/data/moneyball-evaluation-data.csv"
```

```
# Load the datasets directly from GitHub into memory
training = pd.read_csv(training_url)
evaluation = pd.read_csv(evaluation_url)

print("Successfully loaded files from GitHub!")
print(f"Training data shape: {training.shape}")
print(f"Evaluation data shape: {evaluation.shape}")
```

Successfully loaded files from GitHub!

Training data shape: (2276, 17)

Evaluation data shape: (259, 16)

1.1.2 Dataset dimensions

The training and evaluation files should have the same predictor columns. The training file also includes TARGET_WINS.

```
[3]: dataset_dimensions = pd.DataFrame({
    "dataset": ["Training", "Evaluation"],
    "rows": [len(training), len(evaluation)],
    "columns": [training.shape[1], evaluation.shape[1]],
})
display(dataset_dimensions)

print("Training columns:")
print(training.columns.tolist())
print("Evaluation columns:")
print(evaluation.columns.tolist())
```

	dataset	rows	columns
0	Training	2276	17
1	Evaluation	259	16

Training columns:

```
['INDEX', 'TARGET_WINS', 'TEAM_BATTING_H', 'TEAM_BATTING_2B', 'TEAM_BATTING_3B',
'TEAM_BATTING_HR', 'TEAM_BATTING_BB', 'TEAM_BATTING_SO', 'TEAM_BASERUN_SB',
'TEAM_BASERUN_CS', 'TEAM_BATTING_HBP', 'TEAM_PITCHING_H', 'TEAM_PITCHING_HR',
'TEAM_PITCHING_BB', 'TEAM_PITCHING_SO', 'TEAM_FIELDING_E', 'TEAM_FIELDING_DP']
```

Evaluation columns:

```
['INDEX', 'TEAM_BATTING_H', 'TEAM_BATTING_2B', 'TEAM_BATTING_3B',
'TEAM_BATTING_HR', 'TEAM_BATTING_BB', 'TEAM_BATTING_SO', 'TEAM_BASERUN_SB',
'TEAM_BASERUN_CS', 'TEAM_BATTING_HBP', 'TEAM_PITCHING_H', 'TEAM_PITCHING_HR',
'TEAM_PITCHING_BB', 'TEAM_PITCHING_SO', 'TEAM_FIELDING_E', 'TEAM_FIELDING_DP']
```

1.1.3 Identify the dependent and independent variables

The dependent variable, also called the response variable, is TARGET_WINS. It is the number of wins we want to predict.

The independent variables are the baseball performance statistics used to explain or predict TARGET_WINS. INDEX is not an independent variable because it only identifies a row and should not be used in the model.

```
[4]: dependent_variable = "TARGET_WINS"
independent_variables = [
    column for column in training.columns
    if column not in {dependent_variable, "INDEX"}
]

print("Dependent variable:")
print(dependent_variable)
print("Independent variables:")
print(independent_variables)
```

Dependent variable:

TARGET_WINS

Independent variables:

['TEAM_BATTING_H', 'TEAM_BATTING_2B', 'TEAM_BATTING_3B', 'TEAM_BATTING_HR',
 'TEAM_BATTING_BB', 'TEAM_BATTING_SO', 'TEAM_BASERUN_SB', 'TEAM_BASERUN_CS',
 'TEAM_BATTING_HBP', 'TEAM_PITCHING_H', 'TEAM_PITCHING_HR', 'TEAM_PITCHING_BB',
 'TEAM_PITCHING_SO', 'TEAM_FIELDING_E', 'TEAM_FIELDING_DP']

1.1.4 Compare the column names

This check confirms which columns are shared and which columns occur in only one file. The target should be present only in the training data.

```
[5]: all_columns = list(dict.fromkeys([*training.columns, *evaluation.columns]))
column_comparison = pd.DataFrame({
    "in_training": [column in training.columns for column in all_columns],
    "in_evaluation": [column in evaluation.columns for column in all_columns],
}, index=all_columns)
column_comparison.index.name = "column"
display(column_comparison)
```

	in_training	in_evaluation
column		
INDEX	True	True
TARGET_WINS	True	False
TEAM_BATTING_H	True	True
TEAM_BATTING_2B	True	True
TEAM_BATTING_3B	True	True
TEAM_BATTING_HR	True	True
TEAM_BATTING_BB	True	True
TEAM_BATTING_SO	True	True
TEAM_BASERUN_SB	True	True
TEAM_BASERUN_CS	True	True
TEAM_BATTING_HBP	True	True

TEAM_PITCHING_H	True	True
TEAM_PITCHING_HR	True	True
TEAM_PITCHING_BB	True	True
TEAM_PITCHING_SO	True	True
TEAM_FIELDING_E	True	True
TEAM_FIELDING_DP	True	True

1.1.5 Inspect the structure and data types

The variables are expected to be numeric because they represent counts of baseball events or wins.

```
[6]: training.info()
type_summary = pd.DataFrame({
    "class": training.dtypes.astype(str),
    "distinct_values": training.nunique(dropna=True),
})
type_summary.index.name = "column"
display(type_summary)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2276 entries, 0 to 2275
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   INDEX                 2276 non-null   int64
1   TARGET_WINS           2276 non-null   int64
2   TEAM_BATTING_H        2276 non-null   int64
3   TEAM_BATTING_2B       2276 non-null   int64
4   TEAM_BATTING_3B       2276 non-null   int64
5   TEAM_BATTING_HR       2276 non-null   int64
6   TEAM_BATTING_BB       2276 non-null   int64
7   TEAM_BATTING_SO       2174 non-null   float64
8   TEAM_BASERUN_SB       2145 non-null   float64
9   TEAM_BASERUN_CS       1504 non-null   float64
10  TEAM_BATTING_HBP       191 non-null    float64
11  TEAM_PITCHING_H        2276 non-null   int64
12  TEAM_PITCHING_HR       2276 non-null   int64
13  TEAM_PITCHING_BB       2276 non-null   int64
14  TEAM_PITCHING_SO       2174 non-null   float64
15  TEAM_FIELDING_E        2276 non-null   int64
16  TEAM_FIELDING_DP       1990 non-null   float64
dtypes: float64(6), int64(11)
memory usage: 302.4 KB
```

	class	distinct_values
column		
INDEX	int64	2276
TARGET_WINS	int64	108
TEAM_BATTING_H	int64	569

TEAM_BATTING_2B	int64	240
TEAM_BATTING_3B	int64	144
TEAM_BATTING_HR	int64	243
TEAM_BATTING_BB	int64	533
TEAM_BATTING_SO	float64	822
TEAM_BASERUN_SB	float64	348
TEAM_BASERUN_CS	float64	128
TEAM_BATTING_HBP	float64	55
TEAM_PITCHING_H	int64	843
TEAM_PITCHING_HR	int64	256
TEAM_PITCHING_BB	int64	535
TEAM_PITCHING_SO	float64	823
TEAM_FIELDING_E	int64	549
TEAM_FIELDING_DP	float64	144

1.1.6 Check missing values

Missing values need to be understood before modeling. This table shows the count and percentage of missing values in each column.

```
[7]: missing_summary = pd.DataFrame({
    "missing_count": training.isna().sum(),
    "missing_percent": (100 * training.isna().mean()).round(2),
}).sort_values("missing_count", ascending=False)
missing_summary.index.name = "column"
display(missing_summary)
```

	missing_count	missing_percent
column		
TEAM_BATTING_HBP	2085	91.61
TEAM_BASERUN_CS	772	33.92
TEAM_FIELDING_DP	286	12.57
TEAM_BASERUN_SB	131	5.76
TEAM_BATTING_SO	102	4.48
TEAM_PITCHING_SO	102	4.48
TEAM_BATTING_3B	0	0.00
TEAM_BATTING_HR	0	0.00
TEAM_BATTING_BB	0	0.00
TARGET_WINS	0	0.00
TEAM_BATTING_2B	0	0.00
TEAM_BATTING_H	0	0.00
TEAM_PITCHING_H	0	0.00
TEAM_PITCHING_HR	0	0.00
TEAM_PITCHING_BB	0	0.00
TEAM_FIELDING_E	0	0.00
INDEX	0	0.00

1.1.7 Review summary statistics

The minimum, median, mean, and maximum values help show the scale and spread of the variables.

```
[8]: display(training.describe().T)
```

	count	mean	std	min	25%	50% \
column						
INDEX	2276.0	1268.463533	736.349040	1.0	630.75	1270.5
TARGET_WINS	2276.0	80.790861	15.752152	0.0	71.00	82.0
TEAM_BATTING_H	2276.0	1469.269772	144.591195	891.0	1383.00	1454.0
TEAM_BATTING_2B	2276.0	241.246924	46.801415	69.0	208.00	238.0
TEAM_BATTING_3B	2276.0	55.250000	27.938557	0.0	34.00	47.0
TEAM_BATTING_HR	2276.0	99.612039	60.546872	0.0	42.00	102.0
TEAM_BATTING_BB	2276.0	501.558875	122.670862	0.0	451.00	512.0
TEAM_BATTING_SO	2174.0	735.605336	248.526418	0.0	548.00	750.0
TEAM_BASERUN_SB	2145.0	124.761772	87.791166	0.0	66.00	101.0
TEAM_BASERUN_CS	1504.0	52.803856	22.956338	0.0	38.00	49.0
TEAM_BATTING_HBP	191.0	59.356021	12.967123	29.0	50.50	58.0
TEAM_PITCHING_H	2276.0	1779.210457	1406.842930	1137.0	1419.00	1518.0
TEAM_PITCHING_HR	2276.0	105.698594	61.298747	0.0	50.00	107.0
TEAM_PITCHING_BB	2276.0	553.007909	166.357362	0.0	476.00	536.5
TEAM_PITCHING_SO	2174.0	817.730451	553.085031	0.0	615.00	813.5
TEAM_FIELDING_E	2276.0	246.480668	227.770972	65.0	127.00	159.0
TEAM_FIELDING_DP	1990.0	146.387940	26.226385	52.0	131.00	149.0
	75%	max				
column						
INDEX	1915.50	2535.0				
TARGET_WINS	92.00	146.0				
TEAM_BATTING_H	1537.25	2554.0				
TEAM_BATTING_2B	273.00	458.0				
TEAM_BATTING_3B	72.00	223.0				
TEAM_BATTING_HR	147.00	264.0				
TEAM_BATTING_BB	580.00	878.0				
TEAM_BATTING_SO	930.00	1399.0				
TEAM_BASERUN_SB	156.00	697.0				
TEAM_BASERUN_CS	62.00	201.0				
TEAM_BATTING_HBP	67.00	95.0				
TEAM_PITCHING_H	1682.50	30132.0				
TEAM_PITCHING_HR	150.00	343.0				
TEAM_PITCHING_BB	611.00	3645.0				
TEAM_PITCHING_SO	968.00	19278.0				
TEAM_FIELDING_E	249.25	1898.0				
TEAM_FIELDING_DP	164.00	228.0				

In the summary table, TARGET_WINS has a mean of 80.8, a median of 82, and quartiles of 71 and 92. TEAM_BATTING_HBP has 191 non-missing values, compared with 2,276 for TARGET_WINS, so this predictor has substantially fewer recorded observations. The table describes the amount of missing

data but does not show why those values are missing.

1.1.8 Examine the target variable

TARGET_WINS is the response variable for the regression model. We will review its distribution and summary statistics before modeling.

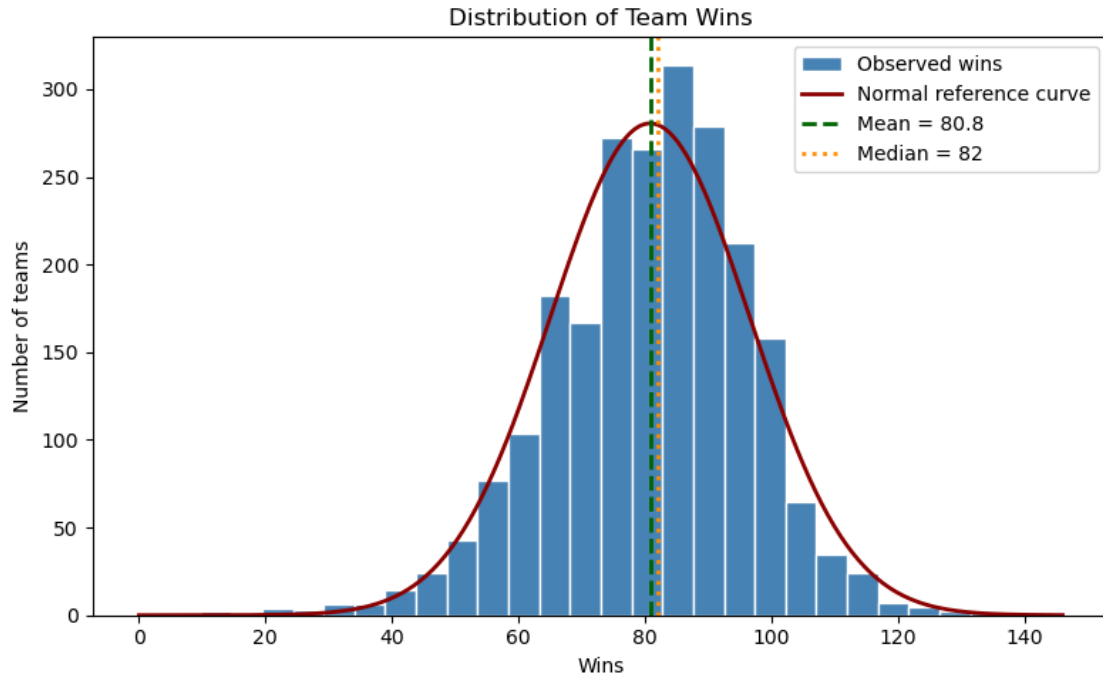
```
[9]: target_summary = training["TARGET_WINS"].describe().to_frame().T
target_summary.index = ["TARGET_WINS"]
display(target_summary)
```

	count	mean	std	min	25%	50%	75%	max
TARGET_WINS	2276.0	80.790861	15.752152	0.0	71.0	82.0	92.0	146.0

The table summarizes TARGET_WINS for 2,276 teams. The mean is about 80.8 wins and the median is 82, so the center is around 81 wins. The middle half of teams recorded 71 to 92 wins, while the full range is 0 to 146 wins (standard deviation: about 15.8). The histogram below shows the shape of this distribution.

```
[10]: wins = training["TARGET_WINS"].dropna()
bin_edges = np.histogram_bin_edges(wins, bins=30)
bin_width = np.diff(bin_edges).mean()
normal_x = np.linspace(wins.min(), wins.max(), 300)
normal_mean = wins.mean()
normal_median = wins.median()
normal_std = wins.std(ddof=1)
normal_counts = (
    len(wins) * bin_width / (normal_std * np.sqrt(2 * np.pi))
    * np.exp(-0.5 * ((normal_x - normal_mean) / normal_std) ** 2)
)

plt.figure(figsize=(8, 5))
plt.hist(wins, bins=bin_edges, color="steelblue", edgecolor="white",
        label="Observed wins")
plt.plot(normal_x, normal_counts, color="darkred", linewidth=2, label="Normal_
        reference curve")
plt.axvline(normal_mean, color="darkgreen", linestyle="--", linewidth=2,
        label=f"Mean = {normal_mean:.1f}")
plt.axvline(normal_median, color="darkorange", linestyle=":", linewidth=2,
        label=f"Median = {normal_median:.0f}")
plt.title("Distribution of Team Wins")
plt.xlabel("Wins")
plt.ylabel("Number of teams")
plt.legend()
plt.tight_layout()
plt.show()
```



Most teams are concentrated around 70–100 wins, with the highest bars near 80–85 wins. The red curve shows the counts expected from a normal distribution with the same mean and standard deviation as `TARGET_WINS`; the bars broadly follow its central shape but differ in places, especially toward the tails. This is a visual reference only. The regression normality assumption should be assessed using model residuals, not the response histogram alone.

1.1.9 Explore relationships between selected predictors and wins

The following plots compare team batting home runs and walks with `TARGET_WINS`. The fitted lines summarize the direction of each pairwise linear association.

```
[11]: predictor_columns = ["TEAM_BATTING_HR", "TEAM_BATTING_BB"]
fig, axes = plt.subplots(1, len(predictor_columns), figsize=(12, 5))

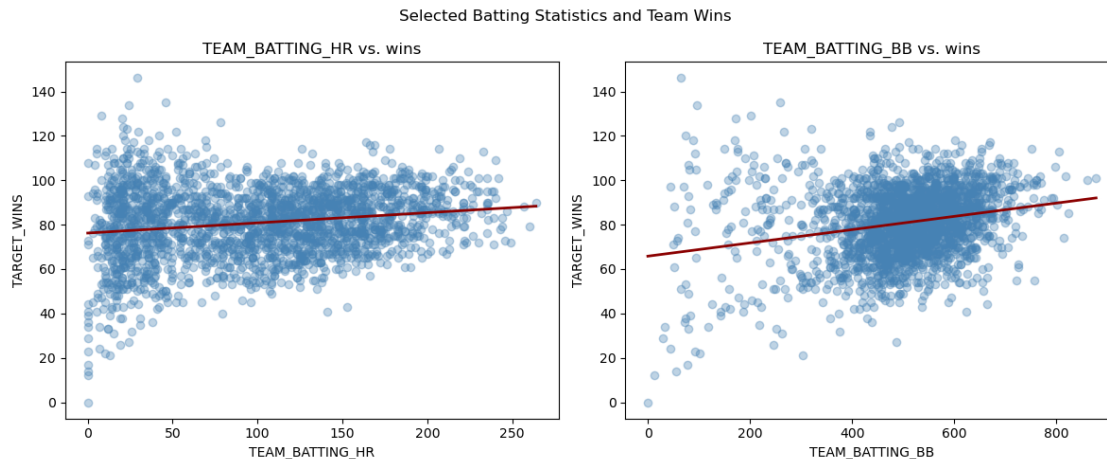
for ax, predictor in zip(axes, predictor_columns):
    plot_data = training[[predictor, "TARGET_WINS"]].dropna()
    ax.scatter(plot_data[predictor], plot_data["TARGET_WINS"], alpha=0.35,
    ↪color="steelblue")
    slope, intercept = np.polyfit(plot_data[predictor],
    ↪plot_data["TARGET_WINS"], 1)
    trend_x = np.linspace(plot_data[predictor].min(), plot_data[predictor].
    ↪max(), 100)
    ax.plot(trend_x, slope * trend_x + intercept, color="darkred", linewidth=2)
    ax.set_title(f"{predictor} vs. wins")
    ax.set_xlabel(predictor)
```

```

ax.set_ylabel("TARGET_WINS")

fig.suptitle("Selected Batting Statistics and Team Wins")
fig.tight_layout()
plt.show()

```



Both plots show a positive pairwise association: teams with more batting home runs or walks tend to have more wins on average. The wide vertical spread at many predictor values shows that neither statistic alone explains wins; other batting, pitching, and fielding factors also matter. These trend lines describe association, not causation, and do not control for the other predictors that will be considered in the regression models.

1.1.10 Correlations among wins and numeric statistics

The heatmap summarizes pairwise Pearson correlations for `TARGET_WINS` and the numeric baseball statistics. `INDEX` is excluded because it is only an identifier.

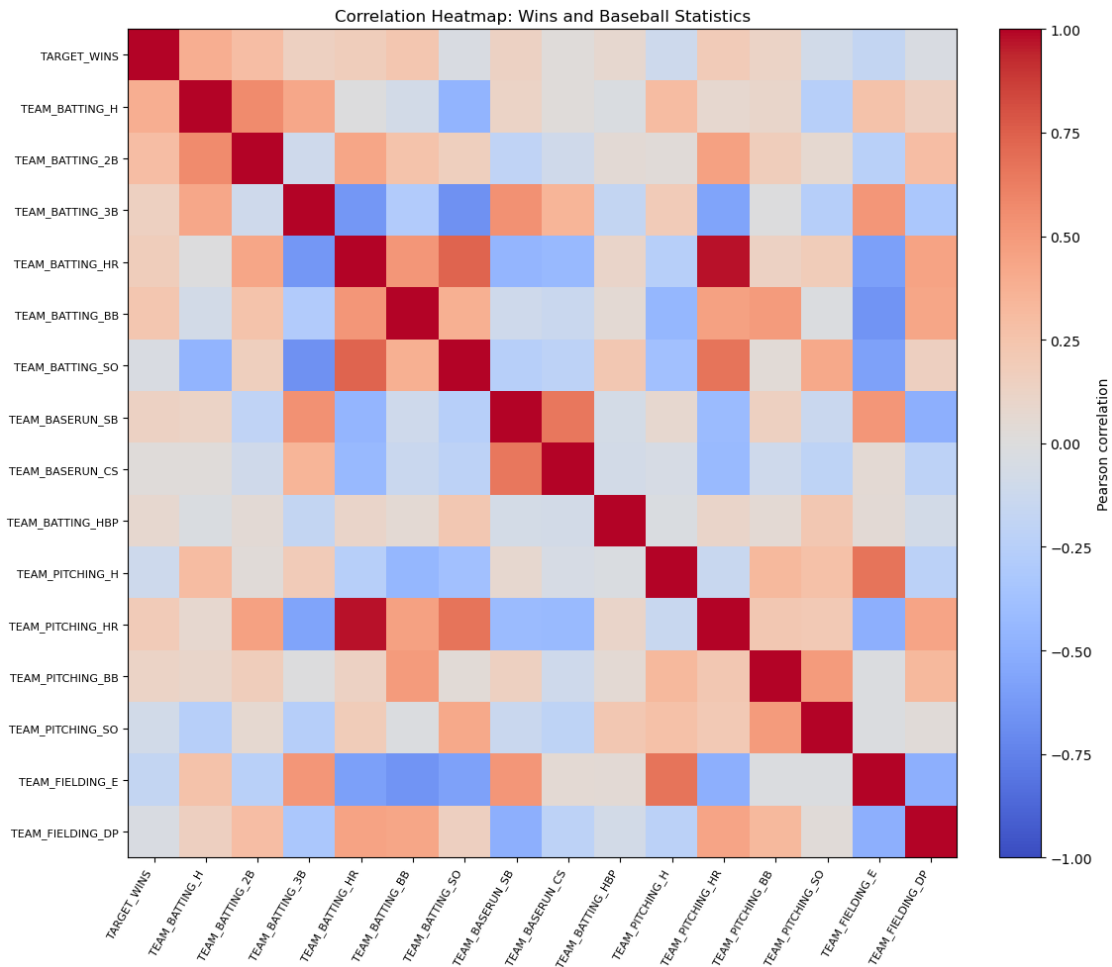
```

[12]: correlation_data = training.select_dtypes(include="number").
      ↪ drop(columns="INDEX", errors="ignore")
      correlation_matrix = correlation_data.corr()

fig, ax = plt.subplots(figsize=(12, 10))
image = ax.imshow(correlation_matrix, cmap="coolwarm", vmin=-1, vmax=1)
ax.set_xticks(np.arange(len(correlation_matrix.columns)))
ax.set_yticks(np.arange(len(correlation_matrix.columns)))
ax.set_xticklabels(correlation_matrix.columns, rotation=60, ha="right",
      ↪ fontsize=8)
ax.set_yticklabels(correlation_matrix.columns, fontsize=8)
ax.set_title("Correlation Heatmap: Wins and Baseball Statistics")
colorbar = fig.colorbar(image, ax=ax, fraction=0.046, pad=0.04)
colorbar.set_label("Pearson correlation")

```

```
fig.tight_layout()
plt.show()
```



Red cells indicate positive correlations and blue cells indicate negative correlations; stronger colors represent stronger linear associations. The **TARGET_WINS** row/column shows mostly modest correlations with individual statistics, while some predictors are strongly correlated with one another. Those predictor to predictor relationships are worth considering when interpreting a multiple regression because they can indicate multicollinearity. Correlations use available pairs of values, so the number of observations can differ across cells when data are missing. Correlation does not establish causation.

1.1.11 Section 1 conclusion

EDA findings: **TARGET_WINS** is centered near 81 wins (mean 80.8; median 82). The scatter plots show positive unadjusted relationships between wins and both batting home runs and walks. The heatmap shows some predictors are correlated with each other. **TEAM_BATTING_HBP** has only 191 observed values, and **TEAM_PITCHING_H** has a maximum of 30,132 compared with a median of 1,518.

Datasets

- Training: `training_url` (2,276 rows, 17 columns), loaded in Python as `training`; includes the response variable `TARGET_WINS`.
- Evaluation: `evaluation_url` (259 rows, 16 columns), loaded in Python as `evaluation`; has the same predictor columns as training but no `TARGET_WINS` values.

Both files contain team level batting, baserunning, pitching, and fielding statistics. `INDEX` is a row identifier and should not be used as a predictor.

1.2 2. Data Preparation

This section handles the structural issues identified during data exploration, including severe missingness, extreme outliers, and multicollinearity. The transformations below prepare the ‘training’ and ‘evaluation’ data for multiple linear regression while maintaining strict data leakage prevention by calculating all transformation metrics (medians, quantiles) exclusively on the training set.

1.2.1 Handling Missing Values and Creating Indicators

Removing Variables: `TEAM_BATTING_HBP` variable was dropped entirely. With 2,085 missing records (91.61% of the dataset), imputation would introduce overwhelming artificial noise rather than genuine signal.

Creating Missingness Flags Binary indicator flags are created, prior to imputation, for variables with significant missingness (`TEAM_BASERUN_CS` at 33.92% and `TEAM_FIELDING_DP` at 12.57%). Because historical baseball data collection practices varied and statistics were not consistently recorded, these missing values are not random. The structural absence of this data may hold predictive power for the model.

Median Imputation: The remaining features with missing data (such as `TEAM_PITCHING_SO` and `TEAM_BATTING_SO`, each missing 4.48%) are imputed using the median. Median imputation is strictly preferred over the mean for this dataset because extreme maximum values (e.g., `TEAM_PITCHING_H` reaching 30,132) would severely distort a mean calculation.

```
[13]: # Fix missing values and create missingness flags

# Drop HBP due to 91.61% missingness
training = training.drop(columns=['TEAM_BATTING_HBP'])
evaluation = evaluation.drop(columns=['TEAM_BATTING_HBP'])

# Create binary flags for variables with >10% missingness
high_missing_cols = ['TEAM_BASERUN_CS', 'TEAM_FIELDING_DP']

for col in high_missing_cols:
    training[f'FLAG_MISS_{col}'] = training[col].isna().astype(int)
    evaluation[f'FLAG_MISS_{col}'] = evaluation[col].isna().astype(int)

# Identify remaining predictors with missing values (excluding TARGET_WINS and
↳ INDEX)
```

```

features_to_impute = [col for col in training.columns if training[col].isna().
    ↪any() and not col.startswith('FLAG_')]

# Calculate medians ONLY from the training set to prevent data leakage
train_medians = training[features_to_impute].median()

# Impute using the calculated training medians
for col in features_to_impute:
    training[col] = training[col].fillna(train_medians[col])
    evaluation[col] = evaluation[col].fillna(train_medians[col])

# Verify missing values are resolved
print("Missing values remaining in Training:", training.isna().sum().sum())
print("Missing values remaining in Evaluation:", evaluation.isna().sum().sum())

```

Missing values remaining in Training: 0
 Missing values remaining in Evaluation: 0

1.2.2 Combining Variables (Feature Engineering)

Deriving Singles: The TEAM_BATTING_H variable represents total hits, which inherently includes the counts of doubles (TEAM_BATTING_2B), triples (TEAM_BATTING_3B), and home runs (TEAM_BATTING_HR). Keeping total hits in the model alongside the extra-base hit variables introduces perfect multicollinearity. A new variable, TEAM_BATTING_1B (Singles), was created by subtracting extra-base hits from total hits. Total hits (TEAM_BATTING_H) was then removed to ensure the regression model could assign distinct, independent weights to different hit types.

```

[14]: # Combine variables: Derive Singles to fix multicollinearity

# Singles = Total Hits - (Doubles + Triples + Home Runs)
training['TEAM_BATTING_1B'] = training['TEAM_BATTING_H'] - (
    training['TEAM_BATTING_2B'] +
    training['TEAM_BATTING_3B'] +
    training['TEAM_BATTING_HR']
)

evaluation['TEAM_BATTING_1B'] = evaluation['TEAM_BATTING_H'] - (
    evaluation['TEAM_BATTING_2B'] +
    evaluation['TEAM_BATTING_3B'] +
    evaluation['TEAM_BATTING_HR']
)

# Drop TEAM_BATTING_H to finalize the isolation of singles and prevent_
↪multicollinearity
training = training.drop(columns=['TEAM_BATTING_H'])
evaluation = evaluation.drop(columns=['TEAM_BATTING_H'])

```

1.2.3 Treating Extreme Values and Mathematical Transformations

Winsorization (Capping): The dataset contains physiologically impossible outliers, such as a maximum of 30,132 for TEAM_PITCHING_H (compared to a median of 1,518) and 19,278 for TEAM_PITCHING_SO. Rather than dropping these rows and sacrificing valuable historical data, the numeric variables are winsorized at the 1st and 99th percentiles. This puts the extreme data into manageable “buckets,” capping reporting errors while retaining the underlying observation.

Logarithmic Transformations: Heavily right-skewed variables with large variance spreads, such as TEAM_FIELDING_E (which ranges from 65 to 1,898), are log-transformed ($\log(x+1)$). This mathematical transform compresses the long tail, pulls extreme values closer to a normal distribution, and stabilizes variance to better satisfy standard linear regression assumptions.

```
[15]: # Transforms (Bucketing/Winsorization and Log Transforms)

# Define continuous numeric features to cap (exclude targets, identifiers, and
# flags)
numeric_features = [col for col in training.columns if col not in
# ['TARGET_WINS', 'INDEX'] and not col.startswith('FLAG_')]

# Calculate the 1st and 99th percentiles strictly on the training set
train_lower_bounds = training[numeric_features].quantile(0.01)
train_upper_bounds = training[numeric_features].quantile(0.99)

for col in numeric_features:
    # Cap both training and evaluation data using the training bounds
    training[col] = training[col].clip(lower=train_lower_bounds[col],
# upper=train_upper_bounds[col])
    evaluation[col] = evaluation[col].clip(lower=train_lower_bounds[col],
# upper=train_upper_bounds[col])

# Apply Log Transformation to highly right-skewed variables (e.g.,
# TEAM_FIELDING_E)
if 'TEAM_FIELDING_E' in training.columns:
    # Use np.log1p (log(1+x)) to gracefully handle any potential zeroes, though
# min is 65
    training['LOG_TEAM_FIELDING_E'] = np.log1p(training['TEAM_FIELDING_E'])
    evaluation['LOG_TEAM_FIELDING_E'] = np.log1p(evaluation['TEAM_FIELDING_E'])

    # Drop the original un-transformed variable
    training = training.drop(columns=['TEAM_FIELDING_E'])
    evaluation = evaluation.drop(columns=['TEAM_FIELDING_E'])

# Display the finalized structure of the prepared training data
training.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2276 entries, 0 to 2275
Data columns (total 18 columns):
```

#	Column	Non-Null Count	Dtype
0	INDEX	2276 non-null	int64
1	TARGET_WINS	2276 non-null	int64
2	TEAM_BATTING_2B	2276 non-null	float64
3	TEAM_BATTING_3B	2276 non-null	float64
4	TEAM_BATTING_HR	2276 non-null	float64
5	TEAM_BATTING_BB	2276 non-null	float64
6	TEAM_BATTING_SO	2276 non-null	float64
7	TEAM_BASERUN_SB	2276 non-null	float64
8	TEAM_BASERUN_CS	2276 non-null	float64
9	TEAM_PITCHING_H	2276 non-null	int64
10	TEAM_PITCHING_HR	2276 non-null	int64
11	TEAM_PITCHING_BB	2276 non-null	int64
12	TEAM_PITCHING_SO	2276 non-null	float64
13	TEAM_FIELDING_DP	2276 non-null	float64
14	FLAG_MISS_TEAM_BASERUN_CS	2276 non-null	int64
15	FLAG_MISS_TEAM_FIELDING_DP	2276 non-null	int64
16	TEAM_BATTING_1B	2276 non-null	float64
17	LOG_TEAM_FIELDING_E	2276 non-null	float64

dtypes: float64(11), int64(7)
memory usage: 320.2 KB

1.3 3. Build Models

Three multiple linear regression models are built on the prepared training data. Each one answers a different question, and variables were selected manually:

Model	Idea	Why
Model 1: Full baseline	Every prepared predictor	Establishes a benchmark and shows where multicollinearity and counter-intuitive signs appear
Model 2: Reduced	Model 1 minus predictors that are redundant <i>and</i> not significant	Tests whether a simpler model loses any explanatory power
Model 3: Derived variables	Hit types + net walk/strikeout differentials + log transforms	Replaces highly collinear batting/pitching pairs with one combined variable each, so the coefficients can be interpreted

For every model we report the coefficients, their p-values, the variance inflation factor (VIF), and whether the sign matches the theoretical effect given in the assignment. A VIF above 10 is used as the rule of thumb for problematic multicollinearity.

```

[16]: expected_sign = {
    "TEAM_BATTING_1B": "+", "TEAM_BATTING_2B": "+", "TEAM_BATTING_3B": "+",
    "TEAM_BATTING_HR": "+", "TEAM_BATTING_BB": "+", "TEAM_BATTING_SO": "-",
    "TEAM_BASERUN_SB": "+", "TEAM_BASERUN_CS": "-",
    "TEAM_PITCHING_H": "-", "LOG_TEAM_PITCHING_H": "-", "TEAM_PITCHING_HR": "-",
    "TEAM_PITCHING_BB": "-", "TEAM_PITCHING_SO": "+",
    "LOG_TEAM_FIELDING_E": "-", "TEAM_FIELDING_DP": "+",
    "NET_BB": "+", "NET_SO": "+",
    "FLAG_MISS_TEAM_BASERUN_CS": "n/a", "FLAG_MISS_TEAM_FIELDING_DP": "n/a",
}

def fit_model(predictors, data):
    """Fit an OLS model of TARGET_WINS on the given list of predictors."""
    formula = "TARGET_WINS ~ " + " + ".join(predictors)
    return smf.ols(formula, data=data).fit()

def coefficient_table(model):
    """Coefficients with p-values, VIF, and a check against the theoretical
    ↪sign."""
    exog = model.model.exog
    names = model.model.exog_names
    vifs = {name: variance_inflation_factor(exog, i) for i, name in
    ↪enumerate(names)}

    table = pd.DataFrame({
        "coefficient": model.params,
        "std_error": model.bse,
        "p_value": model.pvalues,
        "VIF": pd.Series(vifs),
    }).drop(index="Intercept")

    table["expected_sign"] = [expected_sign.get(v, "n/a") for v in table.index]
    table["actual_sign"] = np.where(table["coefficient"] >= 0, "+", "-")
    table["sign_ok"] = np.where(
        table["expected_sign"] == "n/a", "n/a",
        np.where(table["expected_sign"] == table["actual_sign"], "yes", "NO"),
    )
    table["significant_5pct"] = np.where(table["p_value"] < 0.05, "yes", "no")
    return table.round({"coefficient": 4, "std_error": 4, "p_value": 4, "VIF":
    ↪1})

def fit_statistics(model, name):
    """One-row summary of overall fit for a model."""
    return pd.DataFrame({

```

```

    "predictors": [int(model.df_model)],
    "R2": [round(model.rsquared, 4)],
    "adj_R2": [round(model.rsquared_adj, 4)],
    "RMSE": [round(np.sqrt(model.mse_resid), 3)],
    "F_statistic": [round(model.fvalue, 2)],
    "F_p_value": [f"{model.f_pvalue:.2e}"],
    "AIC": [round(model.aic, 1)],
    "BIC": [round(model.bic, 1)],
}, index=[name])

```

1.3.1 Model 1: Full baseline model

Model 1 uses every predictor produced in Section 2: the four hit types (singles, doubles, triples, home runs), batting walks and strikeouts, stolen bases and caught stealing, the four pitching statistics, log errors, double plays, and the two missingness flags. INDEX and the dropped TEAM_BATTING_HBP are excluded. No variable is removed on statistical grounds here; the point is to see how the model behaves when everything is included.

```

[17]: # model 1: every prepared predictor
model1_predictors = [
    col for col in training.columns
    if col not in ["INDEX", "TARGET_WINS"]
]
model1 = fit_model(model1_predictors, training)

print(model1.summary())
display(fit_statistics(model1, "Model 1: Full"))
display(coefficient_table(model1))

```

```

                                OLS Regression Results
=====
Dep. Variable:                TARGET_WINS    R-squared:                0.322
Model:                        OLS           Adj. R-squared:         0.317
Method:                      Least Squares  F-statistic:              66.95
Date:                        Fri, 02 Oct 2026 Prob (F-statistic):       7.26e-177
Time:                        13:30:41       Log-Likelihood:           -9062.2
No. Observations:            2276          AIC:                   1.816e+04
Df Residuals:                2259          BIC:                   1.826e+04
Df Model:                    16
Covariance Type:             nonrobust
=====
=====
                                coef    std err          t      P>|t|
-----
[0.025    0.975]
-----
Intercept                    88.5222    10.024     8.831     0.000
68.866    108.178

```

TEAM_BATTING_2B	0.0178	0.008	2.339	0.019
0.003 0.033				
TEAM_BATTING_3B	0.1746	0.018	9.945	0.000
0.140 0.209				
TEAM_BATTING_HR	0.1021	0.034	3.031	0.002
0.036 0.168				
TEAM_BATTING_BB	0.0497	0.010	4.917	0.000
0.030 0.070				
TEAM_BATTING_SO	-0.0249	0.006	-4.207	0.000
-0.036 -0.013				
TEAM_BASERUN_SB	0.0246	0.005	4.549	0.000
0.014 0.035				
TEAM_BASERUN_CS	0.0148	0.019	0.762	0.446
-0.023 0.053				
TEAM_PITCHING_H	0.0024	0.001	2.831	0.005
0.001 0.004				
TEAM_PITCHING_HR	-0.0059	0.031	-0.190	0.849
-0.066 0.054				
TEAM_PITCHING_BB	-0.0307	0.008	-3.646	0.000
-0.047 -0.014				
TEAM_PITCHING_SO	0.0210	0.005	4.362	0.000
0.012 0.030				
TEAM_FIELDING_DP	-0.1259	0.015	-8.445	0.000
-0.155 -0.097				
FLAG_MISS_TEAM_BASERUN_CS	5.0275	1.016	4.950	0.000
3.036 7.019				
FLAG_MISS_TEAM_FIELDING_DP	4.2026	1.575	2.669	0.008
1.115 7.291				
TEAM_BATTING_1B	0.0470	0.004	11.106	0.000
0.039 0.055				
LOG_TEAM_FIELDING_E	-15.1486	1.389	-10.904	0.000
-17.873 -12.424				

```

=====
Omnibus:                76.178   Durbin-Watson:                1.123
Prob(Omnibus):          0.000   Jarque-Bera (JB):            205.351
Skew:                   0.039   Prob(JB):                    2.56e-45
Kurtosis:               4.469   Cond. No.                    9.27e+04
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 9.27e+04. This might indicate that there are strong multicollinearity or other numerical problems.

	predictors	R2	adj_R2	RMSE	F_statistic	F_p_value \
Model 1: Full	16	0.3217	0.3168	13.02	66.95	7.26e-177

AIC BIC
Model 1: Full 18158.5 18255.9

	coefficient	std_error	p_value	VIF \
TEAM_BATTING_2B	0.0178	0.0076	0.0194	1.6
TEAM_BATTING_3B	0.1746	0.0176	0.0000	3.0
TEAM_BATTING_HR	0.1021	0.0337	0.0025	55.4
TEAM_BATTING_BB	0.0497	0.0101	0.0000	20.1
TEAM_BATTING_SO	-0.0249	0.0059	0.0000	27.0
TEAM_BASERUN_SB	0.0246	0.0054	0.0000	2.6
TEAM_BASERUN_CS	0.0148	0.0194	0.4464	1.4
TEAM_PITCHING_H	0.0024	0.0009	0.0047	6.0
TEAM_PITCHING_HR	-0.0059	0.0308	0.8490	46.4
TEAM_PITCHING_BB	-0.0307	0.0084	0.0003	12.8
TEAM_PITCHING_SO	0.0210	0.0048	0.0000	17.3
TEAM_FIELDING_DP	-0.1259	0.0149	0.0000	1.7
FLAG_MISS_TEAM_BASERUN_CS	5.0275	1.0156	0.0000	3.1
FLAG_MISS_TEAM_FIELDING_DP	4.2026	1.5747	0.0077	3.7
TEAM_BATTING_1B	0.0470	0.0042	0.0000	3.4
LOG_TEAM_FIELDING_E	-15.1486	1.3892	0.0000	9.5

	expected_sign	actual_sign	sign_ok	significant_5pct
TEAM_BATTING_2B	+	+	yes	yes
TEAM_BATTING_3B	+	+	yes	yes
TEAM_BATTING_HR	+	+	yes	yes
TEAM_BATTING_BB	+	+	yes	yes
TEAM_BATTING_SO	-	-	yes	yes
TEAM_BASERUN_SB	+	+	yes	yes
TEAM_BASERUN_CS	-	+	NO	no
TEAM_PITCHING_H	-	+	NO	yes
TEAM_PITCHING_HR	-	-	yes	no
TEAM_PITCHING_BB	-	-	yes	yes
TEAM_PITCHING_SO	+	+	yes	yes
TEAM_FIELDING_DP	+	-	NO	yes
FLAG_MISS_TEAM_BASERUN_CS	n/a	+	n/a	yes
FLAG_MISS_TEAM_FIELDING_DP	n/a	+	n/a	yes
TEAM_BATTING_1B	+	+	yes	yes
LOG_TEAM_FIELDING_E	-	-	yes	yes

Model 1 interpretation. The model explains about 32% of the variance in wins ($R^2 = 0.322$, adjusted $R^2 = 0.317$) with a residual standard error of about 13 wins, and the F-test ($F = 67$, $p < 0.001$) confirms the predictors jointly explain wins.

Most coefficients have the expected sign. Every hit type is positive, and triples (+0.17 wins each) and home runs (+0.10) are worth more per event than singles (+0.05) or doubles (+0.02). Batting walks and stolen bases are positive, batting strikeouts and walks allowed are negative, and pitching strikeouts are positive. Because errors are log-transformed, the coefficient of -15.1 means a 10% increase in errors is associated with about $15.1 \times \ln(1.1) = 1.4$ fewer wins.

Four coefficients need discussion:

- **TEAM_PITCHING_HR** (-0.006 , $p = 0.85$, $VIF = 46$) has the correct sign but is not significant. It is correlated 0.97 with **TEAM_BATTING_HR**, which also has $VIF = 55$. The two variables carry nearly the same information, so the model cannot separate their effects.
- **TEAM_BASERUN_CS** ($+0.015$, $p = 0.45$) has the wrong sign but is not significant. A third of its values were imputed with the median, which weakens whatever signal it had.
- **TEAM_FIELDING_DP** (-0.13 , $p < 0.001$) is negative, although double plays should help. A plausible explanation is that a double play needs runners on base, so a team that turns many of them is also allowing many baserunners. With the other pitching variables held constant, DP is acting as a proxy for weak run prevention.
- **TEAM_PITCHING_H** ($+0.002$, $p = 0.005$) is positive, although hits allowed should hurt. Even after winsorizing, this variable reaches 7,054, and its largest values are concentrated in the same rows as very high error counts (correlation 0.63 with log errors). These look like early-era or data-quality records. Once errors are held constant, the coefficient is most likely picking up an era or recording effect, not a real baseball relationship.

The batting/pitching pairs for walks ($VIF = 20$ and 13) and strikeouts ($VIF = 27$ and 17) are also highly collinear. The coefficients remain significant, but their standard errors are inflated and the individual values should not be over-interpreted. These issues motivate Models 2 and 3.

1.3.2 Model 2: Reduced model

Model 2 starts from Model 1 and removes the two predictors that were both statistically insignificant and problematic for a clear reason:

- **TEAM_PITCHING_HR** is removed because it is almost a duplicate of **TEAM_BATTING_HR** ($r = 0.97$, $VIF = 46$) and adds nothing once batting home runs are in the model ($p = 0.85$).
- **TEAM_BASERUN_CS** is removed because it is not significant ($p = 0.45$), has the wrong sign, and 34% of its values are imputed. Its missingness flag is kept because that flag *is* significant.

Everything else is kept, including the counter-intuitive **TEAM_FIELDING_DP** and **TEAM_PITCHING_H**. They are strongly significant and have a plausible explanation, so removing them only because of their sign would hide real structure in the data. The question is whether dropping two weak predictors costs any explanatory power.

```
[18]: # model 2: drop the redundant / non-significant predictors identified in model 1
removed_from_model1 = ["TEAM_PITCHING_HR", "TEAM_BASERUN_CS"]
model2_predictors = [col for col in model1_predictors if col not in removed_from_model1]
model2 = fit_model(model2_predictors, training)

print("Removed from Model 1:", removed_from_model1)
print(model2.summary())
display(fit_statistics(model2, "Model 2: Reduced"))
display(coefficient_table(model2))
```

Removed from Model 1: ['TEAM_PITCHING_HR', 'TEAM_BASERUN_CS']

OLS Regression Results

```
=====
Dep. Variable:          TARGET_WINS    R-squared:          0.321
Model:                  OLS           Adj. R-squared:       0.317
```

Method:	Least Squares	F-statistic:	76.51
Date:	Fri, 02 Oct 2026	Prob (F-statistic):	1.28e-178
Time:	13:30:41	Log-Likelihood:	-9062.5
No. Observations:	2276	AIC:	1.816e+04
Df Residuals:	2261	BIC:	1.824e+04
Df Model:	14		
Covariance Type:	nonrobust		

=====

		coef	std err	t	P> t
[0.025	0.975]				

Intercept		89.4337	9.881	9.051	0.000
70.057	108.810				
TEAM_BATTING_2B		0.0184	0.008	2.426	0.015
0.004	0.033				
TEAM_BATTING_3B		0.1742	0.017	10.081	0.000
0.140	0.208				
TEAM_BATTING_HR		0.0945	0.009	10.101	0.000
0.076	0.113				
TEAM_BATTING_BB		0.0495	0.010	4.973	0.000
0.030	0.069				
TEAM_BATTING_SO		-0.0248	0.006	-4.401	0.000
-0.036	-0.014				
TEAM_BASERUN_SB		0.0259	0.005	5.119	0.000
0.016	0.036				
TEAM_PITCHING_H		0.0023	0.001	2.770	0.006
0.001	0.004				
TEAM_PITCHING_BB		-0.0308	0.008	-3.748	0.000
-0.047	-0.015				
TEAM_PITCHING_SO		0.0208	0.005	4.581	0.000
0.012	0.030				
TEAM_FIELDING_DP		-0.1257	0.015	-8.439	0.000
-0.155	-0.096				
FLAG_MISS_TEAM_BASERUN_CS		4.7440	0.945	5.019	0.000
2.890	6.598				
FLAG_MISS_TEAM_FIELDING_DP		4.0009	1.553	2.577	0.010
0.956	7.046				
TEAM_BATTING_1B		0.0470	0.004	11.119	0.000
0.039	0.055				
LOG_TEAM_FIELDING_E		-15.1043	1.360	-11.104	0.000
-17.772	-12.437				

=====

Omnibus:	77.453	Durbin-Watson:	1.124
Prob(Omnibus):	0.000	Jarque-Bera (JB):	210.923
Skew:	0.039	Prob(JB):	1.58e-46
Kurtosis:	4.489	Cond. No.	9.14e+04

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 9.14e+04. This might indicate that there are strong multicollinearity or other numerical problems.

	predictors	R2	adj_R2	RMSE	F_statistic	F_p_value \
Model 2: Reduced	14	0.3215	0.3173	13.016	76.51	1.28e-178

	AIC	BIC
Model 2: Reduced	18155.1	18241.0

	coefficient	std_error	p_value	VIF \
TEAM_BATTING_2B	0.0184	0.0076	0.0153	1.6
TEAM_BATTING_3B	0.1742	0.0173	0.0000	2.9
TEAM_BATTING_HR	0.0945	0.0094	0.0000	4.3
TEAM_BATTING_BB	0.0495	0.0100	0.0000	19.5
TEAM_BATTING_SO	-0.0248	0.0056	0.0000	24.5
TEAM_BASERUN_SB	0.0259	0.0051	0.0000	2.3
TEAM_PITCHING_H	0.0023	0.0008	0.0056	5.9
TEAM_PITCHING_BB	-0.0308	0.0082	0.0002	12.3
TEAM_PITCHING_SO	0.0208	0.0045	0.0000	15.3
TEAM_FIELDING_DP	-0.1257	0.0149	0.0000	1.7
FLAG_MISS_TEAM_BASERUN_CS	4.7440	0.9452	0.0000	2.7
FLAG_MISS_TEAM_FIELDING_DP	4.0009	1.5527	0.0100	3.6
TEAM_BATTING_1B	0.0470	0.0042	0.0000	3.4
LOG_TEAM_FIELDING_E	-15.1043	1.3603	0.0000	9.1

	expected_sign	actual_sign	sign_ok	significant_5pct
TEAM_BATTING_2B	+	+	yes	yes
TEAM_BATTING_3B	+	+	yes	yes
TEAM_BATTING_HR	+	+	yes	yes
TEAM_BATTING_BB	+	+	yes	yes
TEAM_BATTING_SO	-	-	yes	yes
TEAM_BASERUN_SB	+	+	yes	yes
TEAM_PITCHING_H	-	+	NO	yes
TEAM_PITCHING_BB	-	-	yes	yes
TEAM_PITCHING_SO	+	+	yes	yes
TEAM_FIELDING_DP	+	-	NO	yes
FLAG_MISS_TEAM_BASERUN_CS	n/a	+	n/a	yes
FLAG_MISS_TEAM_FIELDING_DP	n/a	+	n/a	yes
TEAM_BATTING_1B	+	+	yes	yes
LOG_TEAM_FIELDING_E	-	-	yes	yes

Model 2 interpretation. Removing the two predictors leaves the adjusted R² unchanged (0.317) and slightly lowers AIC and BIC, so they were contributing nothing. Every remaining predictor is significant at the 5% level. The VIF for TEAM_BATTING_HR falls from about 55 to about 4, which

confirms that its earlier inflation came entirely from its pitching twin. The home-run coefficient is now stable at about +0.09 wins per home run.

The remaining weakness is that the walk and strikeout pairs are still collinear (TEAM_BATTING_SO VIF 25, TEAM_PITCHING_SO 15, TEAM_BATTING_BB 20, TEAM_PITCHING_BB 12). TEAM_FIELDING_DP and TEAM_PITCHING_H keep their counter-intuitive signs for the reasons given under Model 1. Model 2 is kept as a candidate: it is as accurate as Model 1 with fewer variables.

1.3.3 Model 3: Derived-variable model

Model 3 deals with the remaining multicollinearity by combining variables instead of dropping them. Three derived variables are created, and the same formulas are applied to the evaluation data so it is ready for prediction in Section 5:

- **NET_BB = TEAM_BATTING_BB - TEAM_PITCHING_BB**: walks drawn minus walks allowed. Baseball analysts use run and walk *differentials* because winning depends on outperforming the opponent, not on raw totals. One variable with a clear expected sign (+) replaces a collinear pair.
- **NET_SO = TEAM_PITCHING_SO - TEAM_BATTING_SO**: strikeouts recorded by the team's pitchers minus strikeouts suffered by its batters. It is oriented so that higher is better (expected sign +).
- **LOG_TEAM_PITCHING_H = log(TEAM_PITCHING_H)**: hits allowed stays heavily right-skewed even after winsorizing (median 1,518, cap 7,054). The log reduces the leverage of the extreme rows, just as Section 2 did for errors.

The four hit types, stolen bases, log errors, double plays, and both missingness flags are kept from Model 2. TEAM_PITCHING_HR and TEAM_BASERUN_CS stay out for the reasons given above.

Note: in about 35% of rows the batting and pitching values for walks and strikeouts are identical, which is a quirk of how this dataset was built. For those rows NET_BB and NET_SO equal 0, which limits how much signal the differentials can carry.

```
[19]: # model 3: derived variables (same application to training and evaluation data)
def add_model3_features(df):
    """Create the Model 3 derived variables in place and return the data frame.
    ↪ """
    df["NET_BB"] = df["TEAM_BATTING_BB"] - df["TEAM_PITCHING_BB"]
    df["NET_SO"] = df["TEAM_PITCHING_SO"] - df["TEAM_BATTING_SO"]
    df["LOG_TEAM_PITCHING_H"] = np.log(df["TEAM_PITCHING_H"])
    return df

training = add_model3_features(training)
evaluation = add_model3_features(evaluation)

model3_predictors = [
    "TEAM_BATTING_1B", "TEAM_BATTING_2B", "TEAM_BATTING_3B", "TEAM_BATTING_HR",
    "NET_BB", "NET_SO", "TEAM_BASERUN_SB",
    "LOG_TEAM_PITCHING_H", "LOG_TEAM_FIELDING_E", "TEAM_FIELDING_DP",
    "FLAG_MISS_TEAM_BASERUN_CS", "FLAG_MISS_TEAM_FIELDING_DP",
]
```

```

model3 = fit_model(model3_predictors, training)

print(model3.summary())
display(fit_statistics(model3, "Model 3: Derived"))
display(coefficient_table(model3))

# confirm the derived columns exist and are complete in the evaluation data
print("Missing values in evaluation derived columns:",
      evaluation[["NET_BB", "NET_SO", "LOG_TEAM_PITCHING_H"]].isna().sum().
      ↪sum())

```

OLS Regression Results

```

=====
Dep. Variable:          TARGET_WINS    R-squared:                0.313
Model:                  OLS           Adj. R-squared:           0.310
Method:                 Least Squares F-statistic:              86.00
Date:                  Fri, 02 Oct 2026 Prob (F-statistic):      1.12e-174
Time:                  13:30:41       Log-Likelihood:          -9076.3
No. Observations:      2276          AIC:                    1.818e+04
Df Residuals:          2263          BIC:                    1.825e+04
Df Model:              12
Covariance Type:       nonrobust
=====

```

```

=====
                                coef    std err          t      P>|t|
-----
[0.025    0.975]
-----
Intercept                    44.8718    18.772      2.390    0.017
8.059    81.685
TEAM_BATTING_1B              0.0464     0.004    10.657    0.000
0.038     0.055
TEAM_BATTING_2B              0.0155     0.008     1.966    0.049
4.03e-05    0.031
TEAM_BATTING_3B              0.1912     0.017    11.475    0.000
0.159     0.224
TEAM_BATTING_HR              0.0946     0.009    10.758    0.000
0.077     0.112
NET_BB                      0.0213     0.008     2.746    0.006
0.006     0.037
NET_SO                      0.0098     0.004     2.210    0.027
0.001     0.019
TEAM_BASERUN_SB              0.0308     0.005     6.410    0.000
0.021     0.040
LOG_TEAM_PITCHING_H          8.4189     3.206     2.626    0.009
2.132    14.706
LOG_TEAM_FIELDING_E         -16.9676     1.385   -12.248    0.000
-19.684   -14.251

```

TEAM_FIELDING_DP	-0.1079	0.015	-7.404	0.000
-0.136	-0.079			
FLAG_MISS_TEAM_BASERUN_CS	4.8872	0.921	5.309	0.000
3.082	6.692			
FLAG_MISS_TEAM_FIELDING_DP	3.6059	1.556	2.318	0.021
0.555	6.657			

Omnibus:	56.078	Durbin-Watson:	1.102
Prob(Omnibus):	0.000	Jarque-Bera (JB):	126.997
Skew:	0.040	Prob(JB):	2.65e-28
Kurtosis:	4.154	Cond. No.	7.83e+04

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 7.83e+04. This might indicate that there are strong multicollinearity or other numerical problems.

	predictors	R2	adj_R2	RMSE	F_statistic	F_p_value \
Model 3: Derived	12	0.3132	0.3096	13.089	86.0	1.12e-174

	AIC	BIC
Model 3: Derived	18178.6	18253.1

	coefficient	std_error	p_value	VIF \
TEAM_BATTING_1B	0.0464	0.0044	0.0000	3.6
TEAM_BATTING_2B	0.0155	0.0079	0.0494	1.7
TEAM_BATTING_3B	0.1912	0.0167	0.0000	2.7
TEAM_BATTING_HR	0.0946	0.0088	0.0000	3.7
NET_BB	0.0213	0.0078	0.0061	5.8
NET_SO	0.0098	0.0044	0.0272	4.1
TEAM_BASERUN_SB	0.0308	0.0048	0.0000	2.0
LOG_TEAM_PITCHING_H	8.4189	3.2059	0.0087	9.9
LOG_TEAM_FIELDING_E	-16.9676	1.3853	0.0000	9.3
TEAM_FIELDING_DP	-0.1079	0.0146	0.0000	1.6
FLAG_MISS_TEAM_BASERUN_CS	4.8872	0.9205	0.0000	2.5
FLAG_MISS_TEAM_FIELDING_DP	3.6059	1.5559	0.0206	3.5

	expected_sign	actual_sign	sign_ok	significant_5pct
TEAM_BATTING_1B	+	+	yes	yes
TEAM_BATTING_2B	+	+	yes	yes
TEAM_BATTING_3B	+	+	yes	yes
TEAM_BATTING_HR	+	+	yes	yes
NET_BB	+	+	yes	yes
NET_SO	+	+	yes	yes
TEAM_BASERUN_SB	+	+	yes	yes
LOG_TEAM_PITCHING_H	-	+	NO	yes
LOG_TEAM_FIELDING_E	-	-	yes	yes

TEAM_FIELDING_DP	+	-	NO	yes
FLAG_MISS_TEAM_BASERUN_CS	n/a	+	n/a	yes
FLAG_MISS_TEAM_FIELDING_DP	n/a	+	n/a	yes

Missing values in evaluation derived columns: 0

Model 3 interpretation. Model 3 uses 12 predictors instead of 14 and reaches an adjusted R^2 of 0.310, slightly below Model 2's 0.317. RMSE is about 13.1 wins versus 13.0. In exchange, **no predictor has a VIF above 10**, compared with four predictors above 10 in Model 2, and every coefficient is significant at the 5% level.

The derived variables behave as expected. `NET_BB` is positive (+0.021 wins per extra walk of advantage), and `NET_SO` is positive (+0.010 per extra strikeout of advantage). The hit-type coefficients are nearly identical to Models 1 and 2 (singles +0.046, doubles +0.016, triples +0.19, home runs +0.095), which shows those estimates are robust to how the rest of the model is specified. A 10% increase in errors is now associated with about $17.0 \times \ln(1.1) \approx 1.6$ fewer wins.

Two signs are still counter-intuitive, and they are kept deliberately:

- `LOG_TEAM_PITCHING_H` (+8.4) stays positive even after the log transform. This supports the Model 1 explanation that the variable is tracking an era or data-recording effect tied to the extreme-error rows, not a true benefit from allowing hits. Removing it would lower fit, and its sign has been consistent across all three models, so it is kept and flagged. Its coefficient should not be read as causal.
- `TEAM_FIELDING_DP` (−0.11) stays negative in all three models, which is consistent with the “more double plays means more baserunners allowed” explanation.

The missingness flags are positive in all three models: rows where caught stealing or double plays were not recorded average a few more wins than the other predictors would suggest. These flags capture differences between record-keeping eras and are not interpreted causally.

1.3.4 Side-by-side summary of the three models

This table collects the overall fit statistics for the three models. Section 4 uses it, together with residual diagnostics, to choose the final model.

```
[20]: # compare the 3 models on overall fit and multicollinearity
models = {"Model 1: Full": model1, "Model 2: Reduced": model2, "Model 3:
↳Derived": model3}

comparison = pd.concat([fit_statistics(m, name) for name, m in models.items()])
comparison["max_VIF"] = [coefficient_table(m)["VIF"].max() for m in models.
↳values()]
comparison["n_VIF_over_10"] = [(coefficient_table(m)["VIF"] > 10).sum() for m_
↳in models.values()]
comparison["n_wrong_sign"] = [(coefficient_table(m)["sign_ok"] == "NO").sum()_
↳for m in models.values()]
comparison["n_not_significant"] = [(coefficient_table(m)["p_value"] >= 0.05).
↳sum() for m in models.values()]
display(comparison)
```

	predictors	R2	adj_R2	RMSE	F_statistic	F_p_value	\
Model 1: Full	16	0.3217	0.3168	13.020	66.95	7.26e-177	
Model 2: Reduced	14	0.3215	0.3173	13.016	76.51	1.28e-178	
Model 3: Derived	12	0.3132	0.3096	13.089	86.00	1.12e-174	

	AIC	BIC	max_VIF	n_VIF_over_10	n_wrong_sign	\
Model 1: Full	18158.5	18255.9	55.4	6	3	
Model 2: Reduced	18155.1	18241.0	24.5	4	2	
Model 3: Derived	18178.6	18253.1	9.9	0	2	

	n_not_significant
Model 1: Full	2
Model 2: Reduced	0
Model 3: Derived	0

1.3.5 Section 3 conclusion

- **Model 1** (all predictors) sets the benchmark at adjusted R^2 0.317. It has severe multicollinearity (batting and pitching HR VIFs near 50) and two insignificant predictors.
- **Model 2** removes the redundant `TEAM_PITCHING_HR` and the weak `TEAM_BASERUN_CS` with **no loss** in adjusted R^2 , and every remaining predictor is significant. The walk and strikeout pairs are still collinear.
- **Model 3** replaces those pairs with net differentials and logs hits allowed. It gives up about 0.007 in adjusted R^2 in exchange for **all VIFs below 10**, fewer predictors, and coefficients that are easier to interpret.

Across all three models, `TEAM_FIELDING_DP` and `TEAM_PITCHING_H` keep counter-intuitive signs. They are retained because they are consistently significant and have plausible data-related explanations (double plays require baserunners; extreme hits-allowed values sit in early-era or low-quality records). Their coefficients are not read as causal effects. The final choice between the more accurate Model 2 and the more parsimonious, interpretable Model 3 is made in Section 4.

1.4 4. Select Models

This section compares the three candidate models and selects a final model for prediction. The decision considers training sample fit (R^2 , adjusted R^2 , MSE, RMSE, and the F statistic), parsimony, coefficient interpretability, and multicollinearity. Residual and influence diagnostics are then used to identify important limitations before applying the selected model to the evaluation data.

```
[21]: # Select the final model and report assignment-required training metrics
selected_model_name = "Model 3: Derived"
selected_model = models[selected_model_name]

display(comparison.loc[[selected_model_name]])

residuals = selected_model.resid
fitted_values = selected_model.fittedvalues
training_mse = np.mean(residuals ** 2)
```

```

selection_metrics = pd.Series({
    "MSE": training_mse,
    "RMSE": np.sqrt(training_mse),
    "R_squared": selected_model.rsquared,
    "adjusted_R_squared": selected_model.rsquared_adj,
    "F_statistic": selected_model.fvalue,
    "F_p_value": selected_model.f_pvalue,
    "max_VIF": coefficient_table(selected_model)["VIF"].max(),
})
display(selection_metrics.to_frame("Model 3"))

```

	predictors	R2	adj_R2	RMSE	F_statistic	F_p_value	\
Model 3: Derived	12	0.3132	0.3096	13.089	86.0	1.12e-174	

	AIC	BIC	max_VIF	n_VIF_over_10	n_wrong_sign	\
Model 3: Derived	18178.6	18253.1	9.9	0	2	

	n_not_significant
Model 3: Derived	0

	Model 3
MSE	1.703401e+02
RMSE	1.305144e+01
R_squared	3.132036e-01
adjusted_R_squared	3.095617e-01
F_statistic	8.600072e+01
F_p_value	1.119621e-174
max_VIF	9.900000e+00

1.4.1 Why Model 3 Was Selected

Model 1 was not selected because it has the weakest fit of the three candidates (adjusted $R^2 = 0.3168$; RMSE = 13.020), six VIFs above 10, and two predictors that are not significant. Model 2 was not selected even though it has the best training sample fit (adjusted $R^2 = 0.3173$; RMSE = 13.016) because it still has four VIFs above 10, including the batting and pitching walk and strikeout pairs. Those relationships inflate standard errors and make individual coefficients less stable to interpret.

Model 3 is selected as the best balance of fit and interpretability. Its adjusted R^2 (0.3096) is only 0.0077 below Model 2 and its RMSE (13.089) is only 0.073 wins higher, while it reduces the specification to 12 predictors and has no VIF above 10 (maximum VIF = 9.9). All 12 retained predictors are significant at the 5% level.

1.4.2 Residual Diagnostics

The plots assess four regression assumptions: residuals centered around zero, stable residual spread across fitted values, approximate normality, and the presence of observations with unusually high leverage or residuals.

```

[22]: # Residual and influence diagnostics for the selected model
from scipy import stats

influence = selected_model.get_influence()
studentized_residuals = influence.resid_studentized_internal
leverage = influence.hat_matrix_diag
leverage_cutoff = 2 * (selected_model.df_model + 1) / selected_model.nobs

fig, axes = plt.subplots(2, 2, figsize=(12, 8), constrained_layout=True)

axes[0, 0].scatter(fitted_values, residuals, alpha=0.6, color="steelblue")
axes[0, 0].axhline(0, color="black", linewidth=1)
axes[0, 0].set(title="Residuals vs. fitted values", xlabel="Fitted wins",
    ↪ylabel="Residuals")

(quantiles, ordered_residuals), (qq_slope, qq_intercept, _) = stats.
    ↪probplot(residuals, dist="norm", fit=True)
axes[0, 1].scatter(quantiles, ordered_residuals, alpha=0.6, color="steelblue")
axes[0, 1].plot(quantiles, qq_slope * quantiles + qq_intercept, color="black",
    ↪linewidth=1)
axes[0, 1].set(title="Normal quantile plot", xlabel="Theoretical quantiles",
    ↪ylabel="Ordered residuals")

axes[1, 0].scatter(fitted_values, np.sqrt(np.abs(studentized_residuals)),
    ↪alpha=0.6, color="steelblue")
axes[1, 0].set(title="Scale location plot", xlabel="Fitted wins",
    ↪ylabel="Sqrt(|studentized residual|)")

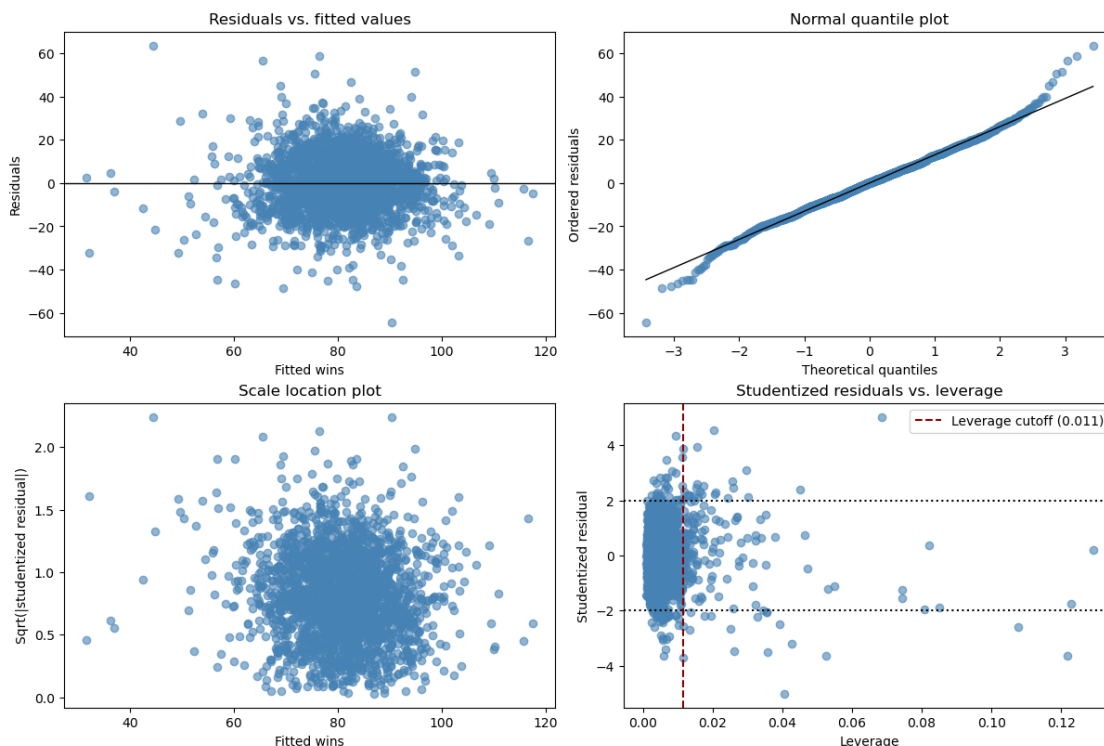
axes[1, 1].scatter(leverage, studentized_residuals, alpha=0.6,
    ↪color="steelblue")
axes[1, 1].axvline(leverage_cutoff, color="darkred", linestyle="--",
    ↪label=f"Leverage cutoff ({leverage_cutoff:.3f})")
axes[1, 1].axhline(2, color="black", linestyle=":")
axes[1, 1].axhline(-2, color="black", linestyle=":")
axes[1, 1].set(title="Studentized residuals vs. leverage", xlabel="Leverage",
    ↪ylabel="Studentized residual")
axes[1, 1].legend()

plt.show()

diagnostics = pd.DataFrame({
    "fitted_wins": fitted_values,
    "residual": residuals,
    "studentized_residual": studentized_residuals,
    "leverage": leverage,
})

```

```
display(diagnostics.reindex(diagnostics["studentized_residual"].abs().
↪sort_values(ascending=False).head(10).index))
```



	fitted_wins	residual	studentized_residual	leverage
1341	44.453639	63.546361	5.030387	0.068519
1827	90.287246	-64.287246	-5.014185	0.040502
2011	76.303006	58.696994	4.530686	0.020287
1819	65.486368	56.513632	4.338010	0.009349
298	94.752809	51.247191	3.946049	0.015512
426	75.423992	50.576008	3.886644	0.011595
858	69.367188	-48.367188	-3.717003	0.011649
2030	83.549217	-47.549217	-3.643677	0.005964
2232	56.687761	-44.687761	-3.643528	0.121931
1824	60.191775	-46.191775	-3.625536	0.052498

The residuals are centered near zero across the fitted range, and the residuals versus fitted plot does not show a strong curved pattern. The scale location plot suggests broadly similar spread across most fitted values, although a small number of observations have much larger residuals. The normal quantile plot is close to the fitted reference line near the center but departs in both tails, indicating that the residual distribution has more extreme values than a normal distribution. Several observations exceed the $|2|$ studentized residual guide, and some have leverage above the dashed cutoff. These cases should be reported as a model limitation and reviewed before final submission; they are not removed automatically because they may represent legitimate historical teams or recording differences.

1.4.3 Section 4 Conclusion

Model 3 is the selected model. It gives up only a small amount of training sample fit relative to Model 2 while using fewer predictors and avoiding VIF values above 10. Its adjusted R^2 is 0.3096, its RMSE is 13.089 wins, and its overall F statistic is statistically significant.

The residual diagnostics support using the model with caution. Residuals are generally centered around zero, but the normal quantile plot shows heavier tails and several observations have large residuals or high leverage. These limitations should be acknowledged in the final report.

1.5 5. Predictions for the evaluation data

The selected Model 3 will be applied to the evaluation dataset to generate predictions for TARGET_WINS. The evaluation data was prepared using the same transformations and derived variables used for the training data.

```
[23]: # Predictions for the evaluation data using model 3
evaluation_predictions = selected_model.predict(evaluation)

print("Number of evaluation observations:", len(evaluation))
print("Number of predictions:", len(evaluation_predictions))
print("Missing predictions:", evaluation_predictions.isna().sum() )

display(evaluation_predictions.head(10))
```

Number of evaluation observations: 259

Number of predictions: 259

Missing predictions: 0

```
0    65.604464
1    65.912984
2    73.108356
3    86.410684
4    73.808707
5    67.008178
6    74.003271
7    74.920448
8    73.859880
9    75.532396
dtype: float64
```

```
[24]: # add predictions to the evaluation dataset
evaluation_results= evaluation[["INDEX"]].copy()
evaluation_results["PREDICTED_WINS"] = evaluation_predictions

display(evaluation_results.head(10))
```

```
   INDEX  PREDICTED_WINS
0      9      65.604464
1     10      65.912984
```

2	14	73.108356
3	47	86.410684
4	60	73.808707
5	63	67.008178
6	74	74.003271
7	83	74.920448
8	98	73.859880
9	120	75.532396

2 Summary of predicted wins

```
prediction_summary= evaluation_results[“PREDICTED_WINS”].describe()
display(prediction_summary.to_frame(“Predicted Wins”))
```

The selected Model 3 was used to generate predictions for all 259 observations in the evaluation dataset. The predicted number of wins ranged from 41.76 to 109.52, with an average predicted value of 80.34 wins.

```
[25]: # Export evaluation predictions to CSV for submission
evaluation_results.to_csv("moneyball_evaluation_predictions.csv", index=False)
print("Predictions successfully exported to CSV!")
```

Predictions successfully exported to CSV!

3 Appendix: Python Code

Analysis performed using Python statsmodels/pandas; code provided in Python per standard of data science workflows.